Protocolo Completo — Teste do Simulador TVDE

Guia exaustivo para executar um teste do simulador do zero, recolher dados e enviá-los para análise.

O que preciso para analisar o panorama global

#	Dado	Como obter	Formato
1	**Resultado do simulador**	Ficheiro `logs/simulator_result_*.txt` após Ctrl+C	Copiar conteúdo ou colar no chat
2	**Stripe Dashboard**	Logs de API em developers.stripe.com → Logs	Screenshot ou copiar lista de requests (status, endpoint, hora)
3	**Stripe CLI**	Output do terminal onde corre `stripe listen`	Copiar as linhas do teste (eventos + respostas 200/erro)
4	**Backend**	Output do terminal onde corre `uvicorn`	Copiar as linhas do teste (requests HTTP)
5	**Base de dados**	Query SQL (ver secção 6)	Resultado da query em texto
6	**unified_payments.csv**	Script de export (ver secção 6)	Ficheiro CSV ou conteúdo

---

PARTE I — Arranque (ordem obrigatória)

Passo 0 — Docker Desktop

1. Abre o **Docker Desktop** (menu Iniciar → Docker Desktop).
2. **Espera 1–2 minutos** até o ícone da baleia aparecer na barra de tarefas e ficar estável.
3. Clica no ícone → deve dizer "Docker Desktop is running".
4. **Nuance:** Na primeira vez após reiniciar o PC, o Docker pode demorar mais. Não avances sem ver "running".

Passo 1 — Base de dados PostgreSQL

Opção A — Script automatizado (recomendado):

1. Abre o **PowerShell** (Windows + escreve "PowerShell" + Enter).
2. Executa:

   ```
   cd C:\dev\APP
   .\scripts\1_start_db.ps1
   ```

3. O script inicia ou cria o contentor e espera 8 s. Quando vir "PostgreSQL pronto", avança.

Opção B — Manual:

1. Se o contentor **já existe:** `docker start ride_postgres`
2. Se é a **primeira vez:** `docker run --name ride_postgres -e POSTGRES_PASSWORD=postgres -e POSTGRES_DB=ride_db -p 5432:5432 -d postgres`
3. **Espera 5–10 segundos** para o PostgreSQL iniciar.
4. Verifica: `docker ps` — deve aparecer `ride_postgres` com status "Up".
5. **Nuance:** Se `docker ps` mostrar o contentor, a BD está pronta. O backend pode falhar se tentar ligar antes dos 5 s.

Passo 2 — Backend (FastAPI)

1. Abre **nova janela** do PowerShell (Ctrl+Shift+N ou File → New Window).
2. Navega e ativa o ambiente:

   ```
   cd C:\dev\APP\backend
   .\venv\Scripts\activate
   ```

3. Verifica o `.env`:
   - `DATABASE_URL=postgresql+psycopg2://postgres:postgres@localhost:5432/ride_db`
   - `ENV=dev`
   - `STRIPE_WEBHOOK_SECRET` — será preenchido no passo 3.
4. Inicia o servidor:

   ```
   uvicorn app.main:app --reload --port 8000
   ```

5. **Espera** até ver:

   ```
   INFO:     Application startup complete.
   ```

6. **Deixa esta janela aberta.** O backend está a correr.
7. **Nuance:** Se der erro de BD, o Docker pode ainda não estar pronto. Espera 10 s e tenta de novo.

Passo 3 — Stripe CLI (webhooks)

1. Abre **nova janela** do PowerShell.
2. Inicia o listener:

   ```
   stripe listen --forward-to localhost:8000/webhooks/stripe
   ```

3. **Espera** até ver "Ready!" e um código `whsec_...`.
4. **IMPORTANTE:** Copia o valor `whsec_...` e cola no `backend/.env` em `STRIPE_WEBHOOK_SECRET=whsec_...`.
5. **Reinicia o backend** (Passo 2): na janela do uvicorn, prima Ctrl+C e volta a executar:

   ```
   uvicorn app.main:app --reload --port 8000
   ```

6. **Deixa esta janela aberta.** O Stripe CLI deve continuar a correr.
7. **Nuance:** Cada vez que reinicias o `stripe listen`, o `whsec_` muda. Tens de atualizar o `.env` e reiniciar o backend.

Passo 4 — Reset da base de dados (antes do teste)

1. Com o backend a correr, abre **nova janela** do PowerShell.
2. Executa o reset (apaga viagens e pagamentos; mantém users/drivers):

   ```
   cd C:\dev\APP
   .\scripts\2_reset_db.ps1
   ```

   Ou manualmente: `curl -X POST http://localhost:8000/dev/reset`
3. **Resultado esperado:** `Reset OK: {"status":"reset_ok"}`
4. **Nuance:** Só funciona com `ENV=dev`. Se der 404, verifica o `.env`.

PARTE II — Execução do teste

Passo 5 — Simulador

1. Na mesma janela (ou nova) do PowerShell, na **raiz do projeto**:

   ```
   cd C:\dev\APP
   python run_simulator.py
   ```

2. Deves ver:

   ```
   Fetching tokens from /dev/seed-simulator...
   Starting 20 passenger bots, 12 driver bots
   API: http://localhost:8000
   Press Ctrl+C to stop.
   ```

3. **Deixa correr** durante o tempo que quiseres (ex.: 3–5 minutos).
4. Para parar: **Ctrl+C**.
5. Deves ver:

   ```
   Stopped.
   Resultado guardado em C:\dev\APP\logs\simulator_result_YYYY-MM-DD_HH-MM-SS.txt
   ```

6. **Timing:** O primeiro ciclo de viagens começa após ~20–120 s (delay aleatório dos bots). Não interrompas imediatamente.

PARTE III — Recolha de dados

Passo 6 — Dados a enviar

Opção A — Script automatizado (recomendado):

1. Após parar o simulador (Ctrl+C), executa:

   ```
   cd C:\dev\APP
   .\scripts\3_collect_data.ps1
   ```

2. O script gera `logs/test_report_YYYY-MM-DD_HH-MM-SS.txt` e `unified_payments.csv`.
3. Copia o conteúdo do relatório e do `simulator_result_*.txt` mais recente.

Opção B — Manual: Executa os comandos abaixo.

6.1 Resultado do simulador

O ficheiro já foi criado. Copia o conteúdo de `C:\dev\APP\logs\simulator_result_*.txt` (o mais recente).

6.2 Base de dados — viagens por estado

```
docker exec ride_postgres psql -U postgres -d ride_db -c "
SELECT status, COUNT(*) AS total
FROM trips
WHERE created_at >= CURRENT_DATE
GROUP BY status
ORDER BY total DESC;
"
```

Copia o output (tabela com status e total).

6.3 Base de dados — resumo completo

```
docker exec ride_postgres psql -U postgres -d ride_db -c "
SELECT
  COUNT(*) AS total_trips,
  COUNT(*) FILTER (WHERE status = 'requested') AS requested,
  COUNT(*) FILTER (WHERE status = 'assigned') AS assigned,
  COUNT(*) FILTER (WHERE status = 'accepted') AS accepted,
  COUNT(*) FILTER (WHERE status = 'arriving') AS arriving,
  COUNT(*) FILTER (WHERE status = 'ongoing') AS ongoing,
  COUNT(*) FILTER (WHERE status = 'completed') AS completed,
  COUNT(*) FILTER (WHERE status = 'cancelled') AS cancelled,
  COUNT(*) FILTER (WHERE status = 'failed') AS failed
FROM trips
WHERE created_at >= CURRENT_DATE;
"
```

6.4 Exportar unified_payments.csv

```
docker exec ride_postgres psql -U postgres -d ride_db -A -F "," -c "
SELECT 'trip_id','trip_status','estimated_price','final_price','trip_created','complete
UNION ALL
SELECT t.id::text, t.status::text, COALESCE(t.estimated_price::text,''), COALESCE(t.fin
FROM trips t
LEFT JOIN payments p ON p.trip_id = t.id
WHERE t.created_at >= CURRENT_DATE
ORDER BY t.created_at;
" > C:\dev\APP\unified_payments.csv
```

O ficheiro fica em `C:\dev\APP\unified_payments.csv`. Podes colar o conteúdo ou anexar.

6.5 Stripe CLI

Na janela onde corre `stripe listen`, seleciona e copia as linhas desde o início do teste até ao fim. Inclui eventos (ex.: `payment_intent.created`) e respostas (`[200] POST ...`).

6.6 Backend

Na janela onde corre `uvicorn`, seleciona e copia as linhas do teste (requests HTTP com status 200, 409, etc.).

6.7 Stripe Dashboard (opcional)

Em **developers.stripe.com** → Logs, filtra pela data/hora do teste e copia ou faz screenshot da lista de requests.

PARTE IV — Resumo do que enviar

Envia num único bloco (ou em partes):

1. **Resultado do simulador** — conteúdo de `logs/simulator_result_*.txt`
2. **Query viagens por estado** — output do comando 6.2
3. **Query resumo** — output do comando 6.3
4. **unified_payments.csv** — conteúdo ou indicação do path
5. **Stripe CLI** — output relevante
6. **Backend** — output relevante
7. **Stripe Dashboard** — se tiveres (opcional)

Checklist rápido

- [ ] Docker a correr
- [ ] `docker start ride_postgres` (ou criar contentor)
- [ ] Backend: `uvicorn app.main:app --reload --port 8000`
- [ ] Stripe CLI: `stripe listen --forward-to localhost:8000/webhooks/stripe`
- [ ] `STRIPE_WEBHOOK_SECRET` no `.env` = valor do stripe listen
- [ ] Reset: `curl -X POST http://localhost:8000/dev/reset`
- [ ] Simulador: `python run_simulator.py` (na raiz do projeto)
- [ ] Ctrl+C para parar
- [ ] Recolher dados (6.1–6.7)

Troubleshooting

Problema	Solução
`docker: command not found`	Docker Desktop não está instalado ou não está no PATH
`connection refused` ao iniciar backend	Espera 10 s após `docker start` e tenta de novo
Webhooks Stripe 401	`STRIPE_WEBHOOK_SECRET` no .env não corresponde ao do `stripe listen`
404 em /dev/reset	`ENV=dev` no .env
Simulador “Failed to fetch tokens”	Backend não está a correr ou /dev/seed-simulator não disponível
`ride_postgres` já existe	Usa `docker start ride_postgres` em vez de `docker run`